

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Introducción a la Programación y Computación 2



Inga. Claudia Liceth Rojas Morales
Ing. Marlon Antonio Pérez Türk
Ing. José Manuel Ruiz Juárez
Ing. Edwin Estuardo Zapeta Gómez
Ing. Dennis Stanley Barrios Gonzalez
Ing. Fernando José Paz González

Tutores de curso:
Isai Eliezer Magdiel Molina Guevara
Eddy Alejandro Murga Barillas
Daniel Eduardo Velásquez Avila
Javier Alejandro Matías Guarcas
Jorge Andrés Mejía Suchite
Carlos Manuel Lima y Lima

PROYECTO 3

OBJETIVO GENERAL

Desarrollar una solución integral que implemente un API que brinde servicios utilizando el Protocolo HTTP bajo el concepto de programación orientada a objetos (POO).

OBJETIVOS ESPECÍFICOS

- Implementar un API a través de lenguaje C# que pueda ser consumida utilizando el protocolo HTTP.
- Utilizar el paradigma de programación orientada a objetos para construir software.
- Utilizar la capa de datos para almacenar información de forma persistente.
- Utilizar mensajes JSON como insumos para la comunicación con el API desarrollado.
- Utilizar expresiones regulares para extraer contenido de texto.

ENUNCIADO

La Superintendencia de Administración Tributaria le ha solicitado construir un software que pueda ser consumido desde Internet como un servicio. Este software recibirá un mensaje con los datos para solicitar la autorización de un Documento Tributario Electrónico (DTE) emitido por un contribuyente y como respuesta emitirá un número único de autorización, este número será un correlativo que iniciará con el valor 1 cada día y no deberá repetirse de nuevo en ese día. La estructura del número de autorización es la siguiente: `yyyymmdd#####`, donde:

yyyy corresponde al año en que se solicita la autorización

mm corresponde al mes en que se solicita la autorización

dd corresponde al día en que se solicita la autorización

corresponde al correlativo precedido por la cantidad de 0's necesarios para mantener el formato: Ej. Para el correlativo 1: 00000001.

El mensaje que el proveedor enviará contendrá la siguiente información:

LUGAR Y FECHA: Guatemala, dd/mm/yyyy hh24:mi

REFERENCIA: A3038

NIT EMISOR: XXXXXXV

NIT RECEPTOR: YYYYYYYV

VALOR: #####.##

IVA: #####.##

TOTAL: #####.##

Reglas y consideraciones del formato para cada mensaje recibido:

- **FECHA:** Fecha en formato dd/mm/yyyy hh24:mi (horas en formato 24 horas – 0 a 23 horas)
- **REFERENCIA:** Código interno enviado por el emisor. Es un texto único de máximo 40 posiciones.
- **NIT EMISOR:** XXXXXXV cada X corresponde a un valor entre 0 ... 9. V debe ser la validación del NIT ingresado. La cantidad de valores X podrá ser desde 1 hasta 20.
- **NIT RECEPTOR:** YYYYYYYV cada Y corresponde a un valor entre 0 ... 9. V debe ser la validación del NIT ingresado. La cantidad de valores Y podrá ser desde 1 hasta 20.
- **VALOR:** Valor real positivo de 2 posiciones decimales.
- **IVA:** Es el resultado de la operación $\text{REDONDEAR}(\text{VALOR} * 0.12, 2)$.
- **TOTAL:** Es el resultado de la operación $\text{VALOR} + \text{IVA}$

NOTA: La validación de NIT utiliza el siguiente algoritmo:

1. Multiplique cada carácter por su posición respectiva (siendo la posición 1 el carácter más a la derecha), excepto el primero.
2. Sume todos los resultados.
3. Obtenga el módulo 11 de la sumatoria.
4. A 11 réstale el resultado obtenido en el punto 3.

5. Calcule el módulo 11 del resultado obtenido en el punto No. 4, si este resultado es 10, entonces el dígito verificador del NIT deberá ser K.

La SAT requiere tener el Registro Tributario Unificado (RTU) de los contribuyentes de Guatemala, en el RTU se almacenarán todos los NITs y nombres de contribuyentes válidos y deberá ser capaz de verificar que los contribuyentes que solicitan la autorización (tanto el emisor como el receptor) existan con un NIT válido.

El programa por desarrollar, luego de recibir el mensaje antes mencionado, deberá almacenar la información necesaria para emitir reportes y realizar consultas. El reporte deberá ser capaz de presentar la siguiente información para un rango de fechas:

FECHA: dd/mm/yyyy

Cantidad total de facturas recibidas: Cantidad total de facturas enviadas al servicio

Listado de errores en facturas recibidas

NIT emisor inválido: cantidad errores

NIT receptor inválido: cantidad errores

NIT emisor inexistente: cantidad inexistentes

NIT receptor inexistente: cantidad inexistentes

IVA mal calculado: cantidad de errores

TOTAL mal calculado: cantidad de errores

REFERENCIA duplicada: cantidad de errores

Cantidad de facturas sin errores: Cantidad total de facturas enviadas sin errores al servicio.

Cantidad de emisores de facturas: Cantidad de distintos NIT's emisores de facturas

Cantidad de receptores de facturas: Cantidad de distintos NIT's receptores de facturas

...

FECHA: dd/mm/yyyy

...

Además, el sistema deberá poder generar un listado de facturas emitidas o recibidas por un NIT de un contribuyente en un rango de fechas dado.

La estructura completa de los mensajes de solicitud y de respuesta está descrita en la siguiente sección.

MENSAJES DE SOLICITUD Y RESPUESTA

Mensajes de solicitud

Para actualizar el RTU se utilizará un mensaje con formato JSON con la siguiente estructura:

```
[
  {"NIT": "7378106", "nombre": "Empresa ABC, S.A."},
  {"NIT": "8338817", "nombre": "Comercial XYZ, S.A."},
  {"NIT": "5123456", "nombre": "Servicios Integrales del Norte"}
]
```

NOTA: Si el NIT es válido y no existe en el RTU se crea un nuevo contribuyente, si el NIT es válido y ya existe se actualizará el contribuyente en el RTU, si el NIT no es válido no tiene ningún efecto sobre el RTU.

El mensaje de entrada para solicitar autorizaciones tendrá un formato JSON.

```
[
  {
    "TIEMPO": "Guatemala, 15/01/2025 15:25 hrs.",
    "REFERENCIA": "A1990",
    "NIT_EMITOR": "7378106",
    "NIT_RECEPTOR": "8338817",
    "VALOR": 100.00,
    "IVA": 12.00,
    "TOTAL": 112.00
  },
  {
    "TIEMPO": "Guatemala, 15/01/2025 15:25 hrs.",
    "REFERENCIA": "A1991",
    "NIT_EMITOR": "7378106",
    "NIT_RECEPTOR": "8338817",
    "VALOR": 200.00,
    "IVA": 24.00,
    "TOTAL": 224.00
  }
]
```

NOTA: El atributo "REFERENCIA" debe ser único.

Mensajes de respuesta

La respuesta a un mensaje de solicitud de actualización de RTU será como la siguiente:

```
{
  "CONTRIBUYENTES_NUEVOS": 2,
  "CONTRIBUYENTES_ACTUALIZADOS": 0,
  "NITS_INVALIDOS": 0
}
```

La respuesta a un archivo de autorizaciones será como la siguiente:

```
{
  "resultadoAutorizaciones": {
    "cantidadAutorizacionesAprobadas": 150,
    "cantidadAutorizacionesRechazadasPorNitEmisorInexistente": 12,
    "cantidadAutorizacionesRechazadasPorNitReceptorInexistente": 8,
    "detalleAprobaciones": [
      {"referencia": "A1990", "codigoAprobacion": 20260921000000001},
      {"referencia": "A1991", "codigoAprobacion": 20260921000000002},
      {"referencia": "A1992", "codigoAprobacion": 20260921000000003}
    ]
  }
}
```

ARQUITECTURA

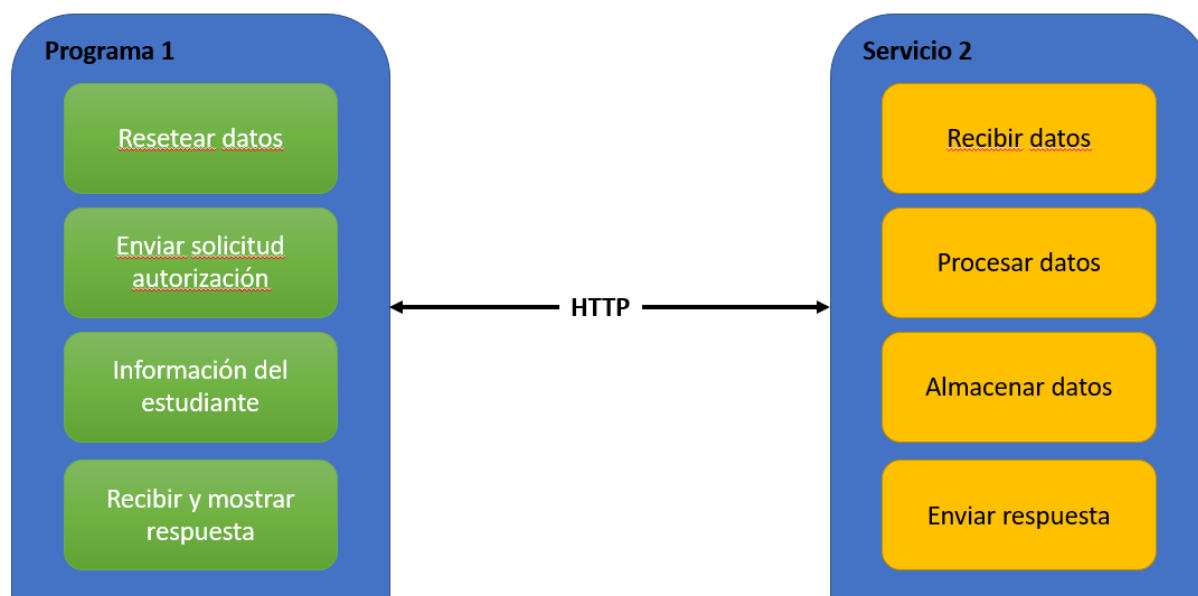


Fig. 1 – Arquitectura general de la aplicación

Programa 1 - Frontend

Este programa consiste en una aplicación Web y consistirá en un simulador de la aplicación principal, contendrá únicamente las funcionalidades necesarias para testear el buen funcionamiento de la API (Servicio 2), en esta aplicación se podrán mostrar los eventos que se procesarán y los datos estadísticos que fueron almacenados.

Para realizar el frontend deberá utilizarse el framework **razor pages**.

Componentes:

- **Enviar mensaje:** Se desplegará una pantalla para gestionar el envío de mensajes pudiendo elegir archivos con extensión json con una o varias solicitudes para actualizar el RTU, o bien, solicitudes de autorización de facturas. La estructura de los mensajes se especifica en la sección MENSAJES DE SOLICITUD Y RESPUESTA.
- **Peticiones:** En este apartado se debe de tener las siguientes opciones:
 - ❖ **Consultar Contribuyente:** Al seleccionar esta opción el usuario podrá elegir entre facturas emitidas o facturas recibidas y luego podrá seleccionar un rango de fechas. Deberá mostrar un reporte indicando por cada fecha la cantidad de facturas autorizadas y el monto total facturado, así como un total facturado en todo el rango de tiempo.
 - ❖ **Resumen de IVA por fecha y NIT:** Al seleccionar esta opción se podrá elegir la fecha por la cual se requiere filtrar y se debe de **mostrar** un resumen de movimientos por NIT en esa fecha dada. Por cada NIT se debe apreciar el IVA emitido y el IVA recibido.
 - ❖ **Resumen por rango de fechas:** Al seleccionar esta opción se podrá elegir un rango de fechas por la cual se requiere filtrar, luego podrá elegir si desea ver

los valores totales o los valores sin IVA. Se deberá **mostrar** por cada fecha el valor autorizado (Total o sin IVA).

- ❖ **Gráfica:** Se deberán crear gráficas para los dos resúmenes anteriores.
 - ❖ **Reporte en PDF:** Deberá ser posible realizar un reporte de todas las peticiones anteriormente descritas en formato pdf.
-
- **Ayuda:** desplegará 2 opciones, una para visualizar información del estudiante y otra para visualizar la documentación del programa.
 - **Opción inicializar:** Esta opción enviará un mensaje a la Api para devolver el sistema al estado inicial, es decir, sin datos.

Servicio 2 - Backend

Este servicio consiste en una API que brindará servicios utilizando el protocolo HTTP, su funcionalidad principal es procesar los datos recibidos del programa 1, luego de procesar los datos es necesario que estos sean almacenados en uno o varios archivos¹, este servicio también tiene la funcionalidad de devolver los mensajes para que sean mostrados como se indica en la sección “Programa 1 – Frontend / Componentes”.

Para la realización de este servicio debe utilizarse **.net core**. El estudiante deberá definir por su propia cuenta los métodos que necesitará para la realización de este servicio. Esto significa que debe implementar tantos métodos como necesite para consumir la API.

NOTA: Durante la calificación de este proyecto, el Servicio 2 podrá ser consumido desde otro cliente, por ejemplo, Postman.

¹ El estudiante puede definir archivos que sean útiles para mostrar los resultados de las solicitudes de autorización atendidas y de la información del RTU.

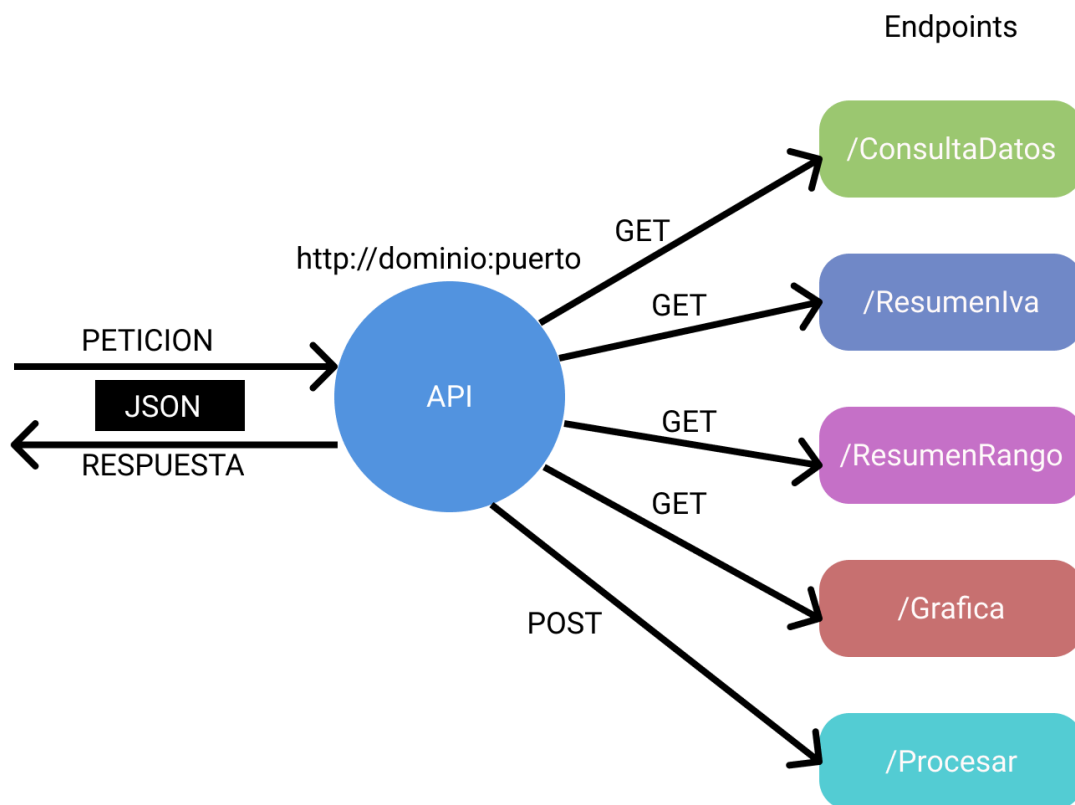


Fig. 2 - Sugerencia de la estructura de un API

CONSIDERACIONES

En las solicitudes de autorización del mensaje de entrada debe descartarse cualquier información que no sea necesaria para la elaboración de las gráficas y archivos de salida, por ejemplo, si las fechas traen nombres de lugares u otra información, éstos no deben ser considerados errores, solamente debe considerarse la información útil para el proyecto.

Se deberá realizar **la implementación utilizando programación orientada a objetos en lenguaje C#**, algoritmos desarrollados por el estudiante e implementación de estructuras a través de Tipos de Dato Abstracto (TDA) propios del estudiante; que permita almacenar la información de los mensajes de entrada y poder interactuar con dicha información. El estudiante deberá abstraer la información y definir qué estructuras implementar que le faciliten la solución. En este proyecto SI está permitido el uso de estructuras propias de C# (ListedLink, List, Queue, Stack, templates, etc.).

Debe utilizarse versionamiento para el desarrollo del proyecto. Se utilizará la plataforma **Github** en la cual se debe crear un repositorio en el que se gestionará el proyecto. Se deben realizar 4 releases o versiones del proyecto (se recomienda realizar una por semana del tiempo disponible) mínimo. **Se deberá agregar a su respectivo auxiliar como colaborador del repositorio.** El último release será el release final y se deberá de realizar antes de entregar el proyecto en la fecha estipulada.

DOCUMENTACIÓN

Para que el proyecto sea calificado, el estudiante deberá entregar la documentación utilizando el formato de ensayo definido para el curso. En el caso del proyecto, el ensayo debe tener entre 4 y 7 páginas de contenido, este máximo no incluye los apéndices o anexos donde se pueden mostrar modelos y diseños utilizados para construir la solución. Es obligatorio incluir el diagrama de clases que modela la solución de software presentada por el estudiante, los diagramas de actividades con los principales algoritmos implementados en la solución y el diagrama de despliegue del software desarrollado.

RESTRICCIONES

- Solo se permitirá la utilización de los IDEs y frameworks indicados en este enunciado.
- Uso obligatorio de programación orientada a objetos (POO) en lenguaje C#. De no cumplir con la restricción, no se tendrá derecho a calificación.
- El nombre del repositorio debe de ser **IPC2_Proj03_202602_#Carnet**.
- El estudiante debe entregar la documentación solicitada para poder optar a la calificación.
- Los mensajes de entrada no podrán modificarse.
- Los mensajes de salida deben llevar la estructura mostrada en el enunciado obligatoriamente.
- Deben existir 4 releases uno por cada semana, de esta manera se corrobora el avance continuo del proyecto. **Se definirá una penalización por cada release faltante.**
- Se calificará el cuarto release almacenado en el repositorio Github. Los cambios realizados después de ese release no se tomarán en cuenta.
- Para dudas concernientes al proyecto se utilizarán los foros en UEDI de manera que todos los estudiantes puedan ver las preguntas y las posteriores respuestas.
- Cualquier caso de copia parcial o total tendrá una nota de 0 y será reportada a Escuela de Ciencias y Sistemas.
- NO HABRÁ PRÓRROGA.

Usuarios GitHub de los auxiliares

- **Sección A - Eddy2109**
- **Sección B - EliezerGG**
- **Sección C - DanielVelAv**
- **Sección D - javiermatg**
- **Sección N - jorgemejia25**
- **Sección P - manuelimal25-dotcom**

ENTREGA

- La entrega será el **22 de octubre** a las 23:59 como máximo.
- La entrega será por medio de la UEDI.
- La documentación debe estar subida en el repositorio en una carpeta separada.
- Para entregar el proyecto en UEDI se deberá subir un archivo de texto con el link del repositorio.